

Authenstickator: a TPM-secured TOTP authenticator app running on a USB stick

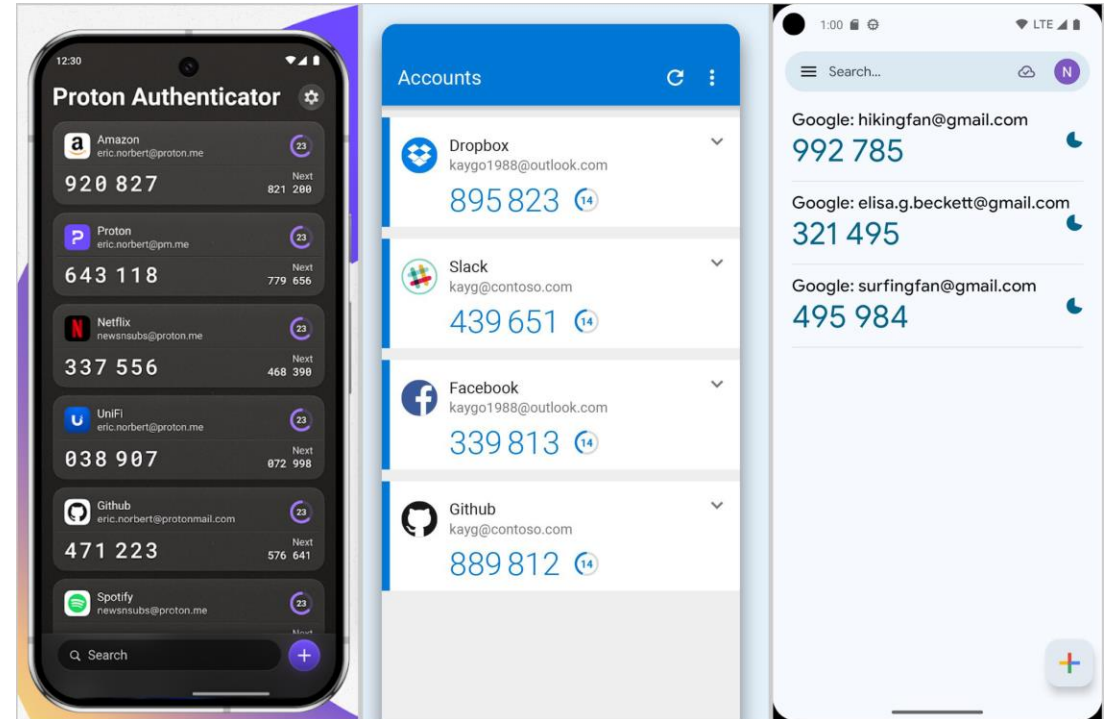
Master graduate: ing. Roland Bálint

Supervisor: conf. dr. ing. Ciprian Pavel Oprea

Technical University of Cluj-Napoca, September, 2026

Two factor authentication

- Passwords are weak by nature.
- Many employ MFA.
- Factors: knowledge, possession, location, time.
- Most common: TOTP-based.
 - *Password* is knowledge.
 - *Mobile phone* running authenticator app generating TOTP codes is possession.



Usual TOTP flow

- A secret is shared, as a QR code
- The user scans the QR code
- The mobile phone decodes the QR code, gets a *Key Uri Format* URI
- The authenticator app parses the URI and stores the secret
- The authenticator app generates codes, which are a function of the secret and current Unix time



otpauth://totp/
Microsoft:rolandtest123456@gmail.com
?secret=**G5HLROANXBF7335S**
&issuer=Microsoft



$$TOTP = HOTP(K, T)$$
$$T = \frac{(CurrentUnixTime - T_0)}{X}$$

$$HOTP(K, C) = \text{Truncate}(\text{HMAC-SHA-1}(K, C))$$



Name: Microsoft:rolandtest123456@gmail.com
Code: 936361
Remaining time: 4

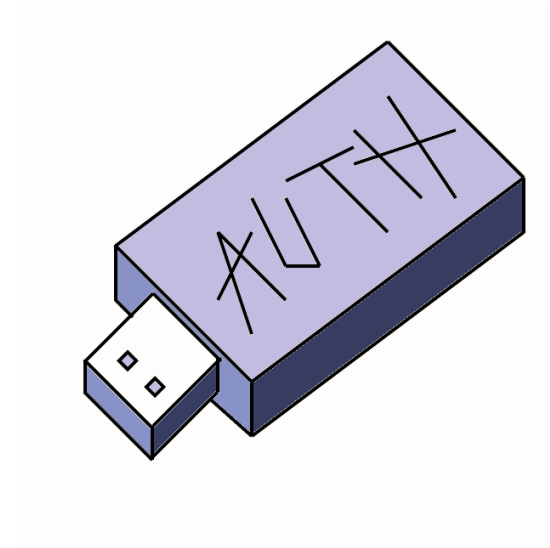
Devices for TOTP code generation

- The authenticator app runs on a phone.
 - Usually in work environments...
 - ...where you do not get a work phone...
 - ...so, you have to use your personal device for work.
- Alternatives: security keys (ex: YubiKey)
 - Dedicated hardware, secure.
 - Can become expensive (~ 70-100 EUR).



TOTP code generation on a USB stick?

- Could bring benefits:
 - **Enforces clear boundaries:** work devices for work, personal devices for personal life.
 - **Cheap:** costs way less than other security solutions.
 - **Reuses old media:** gives a purpose to the old forgotten USB stick.
- No technical limitations to do so.
- Just need a way to do it securely.



Goal: a framework and an implementation

- Framework: a methodology to achieve 3 main goals:
 - **Generate TOTP codes**: the functional requirement, standard algorithm.
 - **Store the secrets safely**: protect at rest and when in use.
 - **Keep the 2FA aspect**: breaking this would make everything useless.
- Implementation: a PoC of the framework
 - **Working app** implementing the framework's suggestion
 - Practicality: user friendliness, cross-compatibility, flexibility
 - Security: the devil lies in the details

Threat model

- **In scope:**

- Protect the secrets at rest, do not expose them while in use
- Make brute-forcing secret extraction as hard as possible
- Do not expose to the internet
- Threat categories: tampering, information disclosure

- **Out of scope:**

- Make the USB read-only, un-copyable, un-formattable
- Detect and report theft and misuse
- Protect against malware having access to RAM while the app is in use
- Threat categories: spoofing, repudiation, denial of service, elevation of privileges

Idea: disk encryption

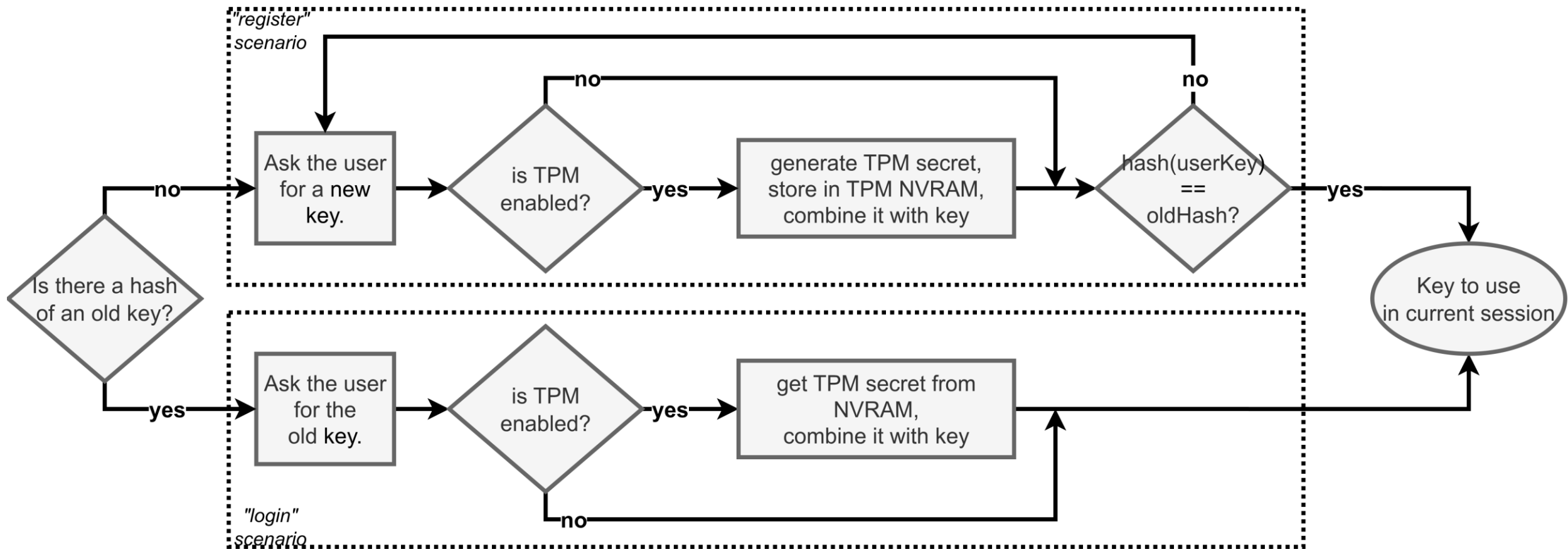
- Applies on the whole media, protection at rest
- Standard algorithms for USB formatting and encryption
 - for Linux, format with Ext4 and encrypt with LUKS2
 - for Windows, format with FAT32 and encrypt with BitLocker
- Requires an **encryption password**
- Drawbacks: cannot enforce, cannot control
- Could store the secret in plaintext... or could we?

Idea: encrypt the storage

- Storage = the application's database = the secrets (+ other non-sensitive info)
- Protects against coffee shop attacks
- Can be enforced by the framework, unlike USB encryption
 - Ask for a **user password**
 - Use it for authentication/ authorization + as an encryption key
- Never store/log user password
 - Instead, hash with Argon2, store hash, check against it at login
- Drawbacks: one more layer, but still susceptible to brute forcing

Idea: use the TPM

- TPM = Trusted Platform Module = dedicated hardware for cryptographic primitives
 - Is able to generate a long, truly random secret => **TPM secret**
 - Is able to store that secret in non-volatile memory (NVRAM)
- Mix it with the user password, in a cryptographically correct manner
 - Expand the user password with PBKDF2, use it as AES password
 - Use the TPM secret as AES salt
 - Makes brute-forcing unfeasible
- Benefit: ties the application to one PC
- Drawback: ties the application to one PC
 - Optional: if not desired by user, a hardcoded salt is used



The app

- Python for backend
 - Many popular cryptography libraries
 - Straight-forward dependency management
- HTML, JavaScript, CSS for frontend
 - the pywebview library wraps all into a desktop application, completely offline
 - Standard technologies, lightweight (just vanilla JS)
- MVC architecture, with Python controllers
 - Every action triggered by View, sent to Controller, delegated to Model
 - Could replace Model or View and keep the other
- Immutable GitHub release, packed on Ubuntu 22, Ubuntu 24 and Windows 10
 - <https://github.com/broland29/authenstickator/releases/tag/v1.0.2>

Register and login views



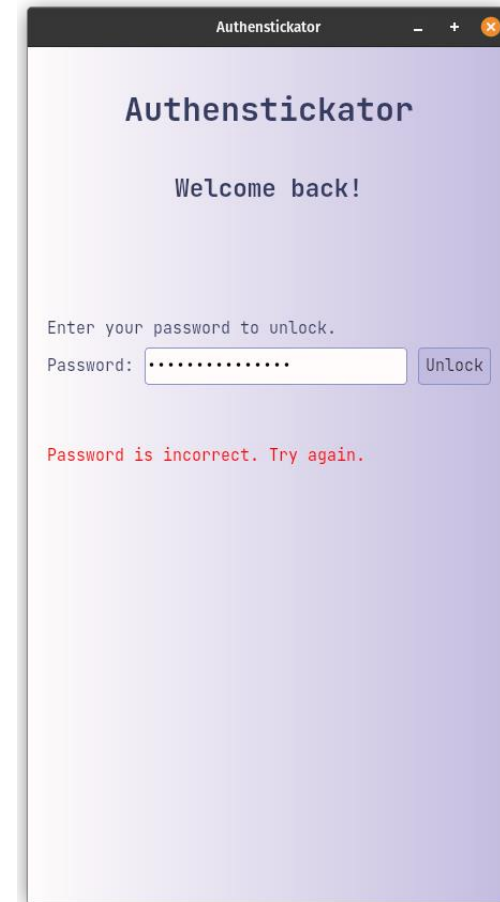
Authenstickator

Welcome!

Please enter a new password.

New password:

This is a screenshot of the 'Authenstickator' application window. The title bar shows the name 'Authenstickator' and standard window controls. The main content area has a light purple gradient background. At the top, the word 'Authenstickator' is displayed in a dark, monospaced font, followed by 'Welcome!' on the next line. Below this, a prompt 'Please enter a new password.' is shown. Underneath the prompt is a label 'New password:' followed by a text input field with a white background and a light blue border. To the right of the input field is a blue button with the text 'Enter' in white.



Authenstickator

Welcome back!

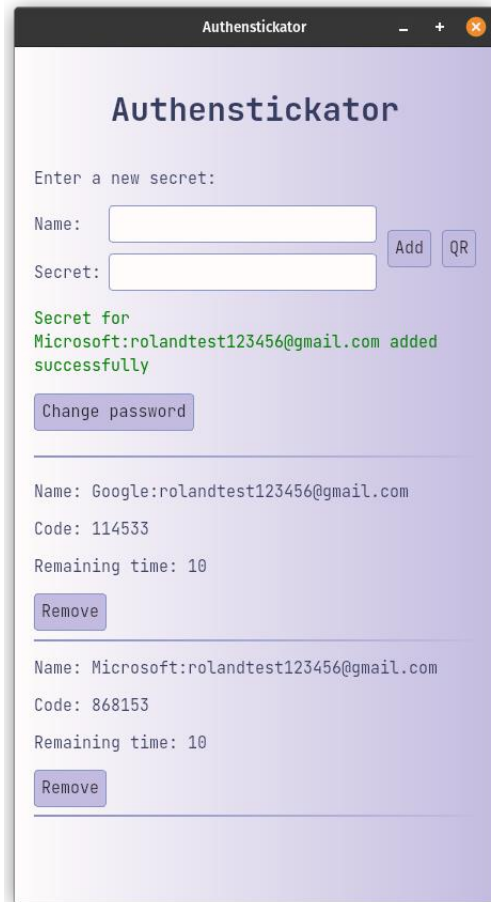
Enter your password to unlock.

Password:

Password is incorrect. Try again.

This is a screenshot of the 'Authenstickator' application window. The title bar shows the name 'Authenstickator' and standard window controls. The main content area has a light purple gradient background. At the top, the word 'Authenstickator' is displayed in a dark, monospaced font, followed by 'Welcome back!' on the next line. Below this, a prompt 'Enter your password to unlock.' is shown. Underneath the prompt is a label 'Password:' followed by a text input field with a white background and a light blue border. To the right of the input field is a blue button with the text 'Unlock' in white. At the bottom of the form, the text 'Password is incorrect. Try again.' is displayed in a red, monospaced font.

TOTP and change password views



Authenstickator

Enter a new secret:

Name:

Secret:

Secret for
Microsoft:rolandtest123456@gmail.com added
successfully

Name: Google:rolandtest123456@gmail.com

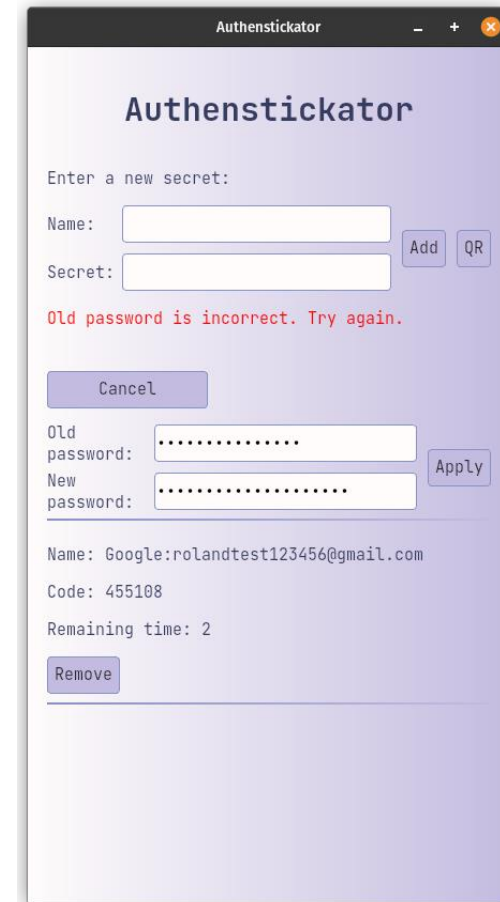
Code: 114533

Remaining time: 10

Name: Microsoft:rolandtest123456@gmail.com

Code: 868153

Remaining time: 10



Authenstickator

Enter a new secret:

Name:

Secret:

Old password is incorrect. Try again.

Old password:

New password:

Name: Google:rolandtest123456@gmail.com

Code: 455108

Remaining time: 2

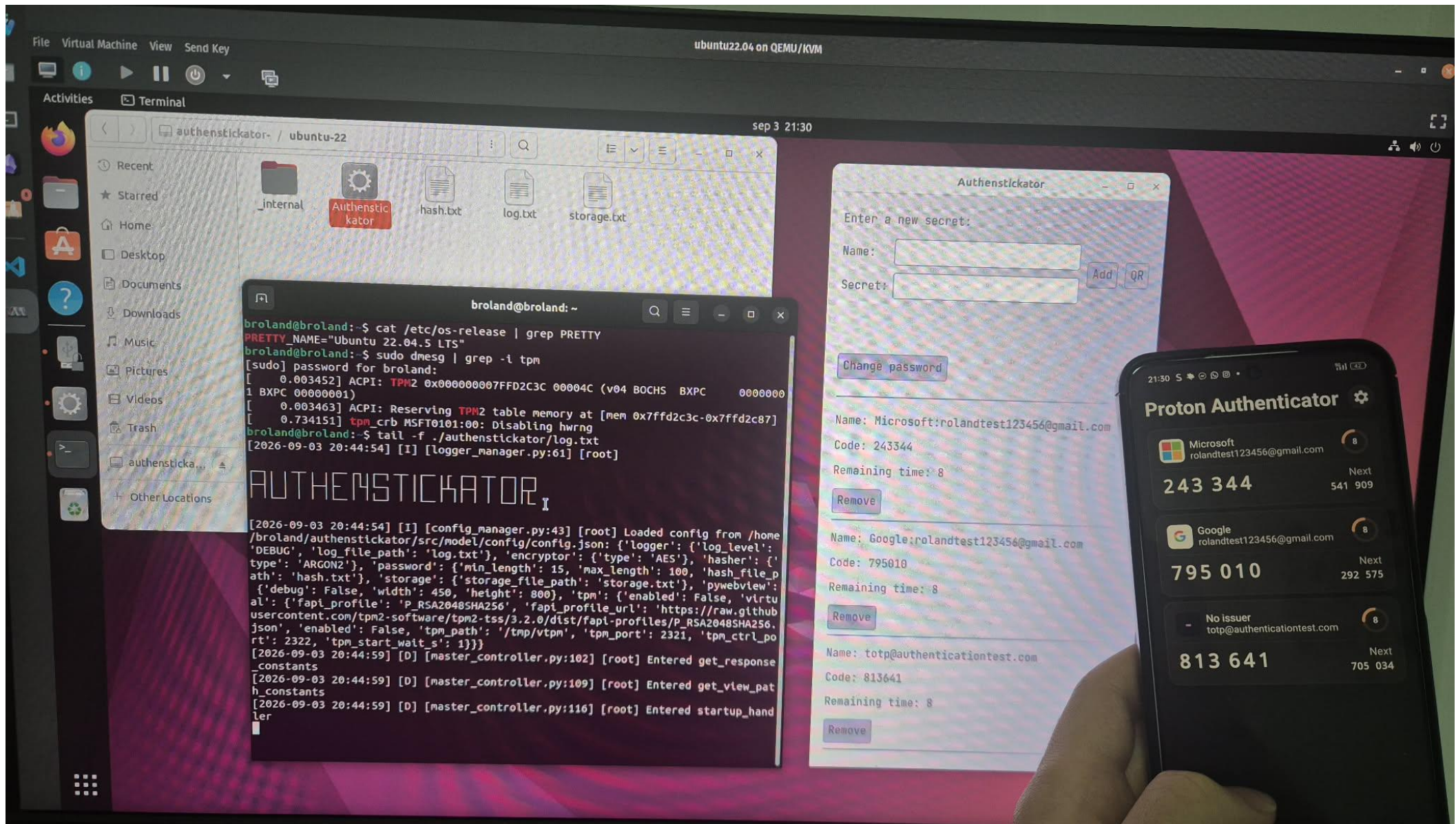
Testing Authenstickator

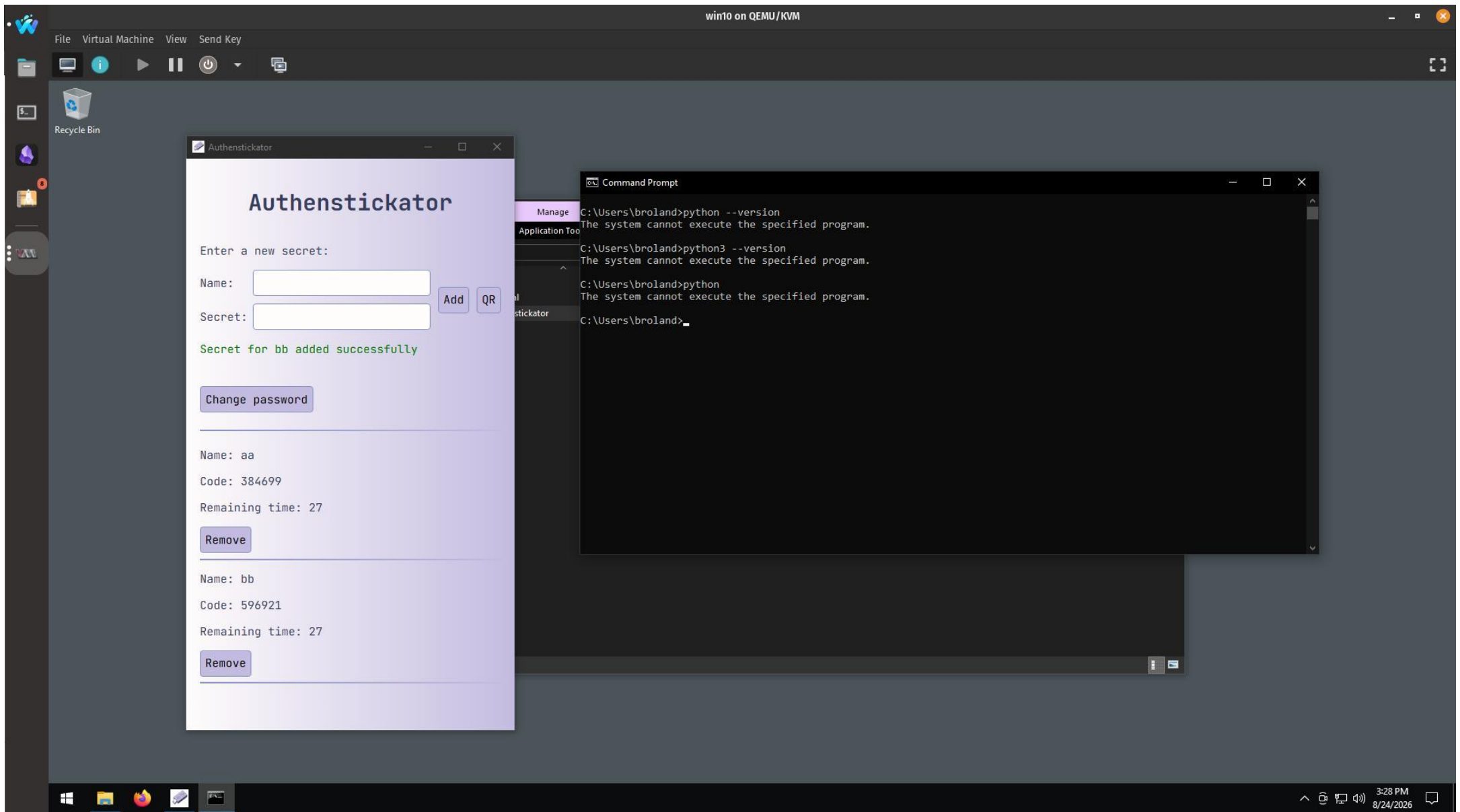
- Manually tested on virtual machines.
 - Ubuntu 22.04 LTS
 - Windows 10
 - Ubuntu 24.04 LTS
- Unit tests for classes of model.
- Tested on Windows 10 machine without Python.

```
(venv) broland@pop-os:~/authenstickator$ python -m pytest
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/broland/authenstickator
collected 21 items

tests/encryptor/test_encryptor.py . [ 4%]
tests/haser/test_haser.py . [ 9%]
tests/qr/test_qr.py ..... [ 33%]
tests/storage/test_storage.py ..... [ 71%]
tests/totp/test_totp.py .... [ 90%]
tests/tpm/test_tpm.py .. [100%]

===== 21 passed in 32.95s =====
(venv) broland@pop-os:~/authenstickator$
```





Quick demo (if time allows)

Conclusions

- The initial goals of the thesis were met.
 - A framework for running TOTP authenticator apps securely on a USB stick was presented.
 - An application called Authenstickator was developed, implementing the theoretical framework.
- The results were validated.
 - The framework is validated by bibliographic research, usage of standard algorithms, cybersecurity best practices, and the implementation, which serves as a proof of concept
 - The application is validated by manual and automated testing.
- **Running a TOTP application on a USB stick securely is possible. It does not provide the same security as dedicated hardware, but it is secure enough for the purpose of two factor authentication.**

Thank you for your attention. Questions?

